

EECS 482

Introduction to operating systems

Managing multiple address spaces

Peter Chen
University of Michigan

Address space protection: summary

- How are address spaces protected?
 - Separation of translation data
- How is translation data protected?
 - Can update translation data only if mode bit set
- How is mode bit protected?
 - Sets/reset mode bit when transitioning from user-level to kernel-level code and back
 - Transitions limited by interrupt vector table
- How is interrupt vector table protected?
- Aside: do administrative processes run in kernel mode or user mode?

How can user process safely switch to kernel mode?

- Faults and interrupts
 - Why are these safe to transfer control to kernel?
- System calls
 - Process management: fork()/exec()
 - Memory management: mmap
 - I/O: open(), close(), read(), write()
 - System management: reboot()
 - ...

System calls

- When you call `cout <<` in your C++ program:
 - `cout` calls `write()`, which executes assembly-language instruction `syscall`
 - `syscall` traps to kernel at **pre-specified** location
 - kernel's `syscall` handler calls kernel's `write()`
- To handle trap to kernel, hardware atomically
 - Sets mode bit to kernel
 - Saves registers, PC, SP
 - Changes SP to kernel stack
 - Changes to kernel's address space
 - Jumps to exception handler
 - **What does this remind you of?**

Passing arguments to system calls

- Which address space (user process's or kernel's) should be used to store the arguments?
- Kernel must carefully check validity of arguments
 - E.g., `write(int fd, void* buf, size_t size)`
 - » Is `fd` a valid descriptor for an open file?
 - » Are all addresses in `[buf, buf+size)` valid?
 - » Are all addresses in `[buf, buf+size)` readable?

Process creation

- Steps
 - Allocate process control block
 - Initialize translation data for new address space
 - Read program image from executable into memory
 - Create initial thread
 - Initialize registers
 - Set mode bit to “user”
 - Jump to start of program
- Need hardware support to combine last few steps (also used to switch from kernel to user process after system call)

Physical memory is a cache for address spaces

- How to divide physical memory among processes?
 - Resource allocation
 - Fairness vs. efficiency
- Global replacement
 - Can evict pages from any process
- Local replacement
 - Can evict pages only from faulting process
 - Must pre-determine how many physical pages each process gets

Average access time

- Performance degrades rapidly as miss rate goes up
 - Avg access time = hit rate * hit time + miss rate * miss time
 - E.g., hit time = .0001 ms; miss time = 10 ms
 - » Average access time (100% hit rate) = .0001 ms
 - » Average access time (99% hit rate) = .100099 ms
 - » Average access time (90% hit rate) = 1.0009 ms
- High miss rate = thrashing
 - E.g., many large processes all actively use a lot of virtual memory

Solutions to thrashing?

- Buy more DRAM
- Change scheduler to reduce thrashing
 - E.g., system has 1 GB memory
 - How to run processes A, B, C, D (each 500 MB)?
 - Why does this help?

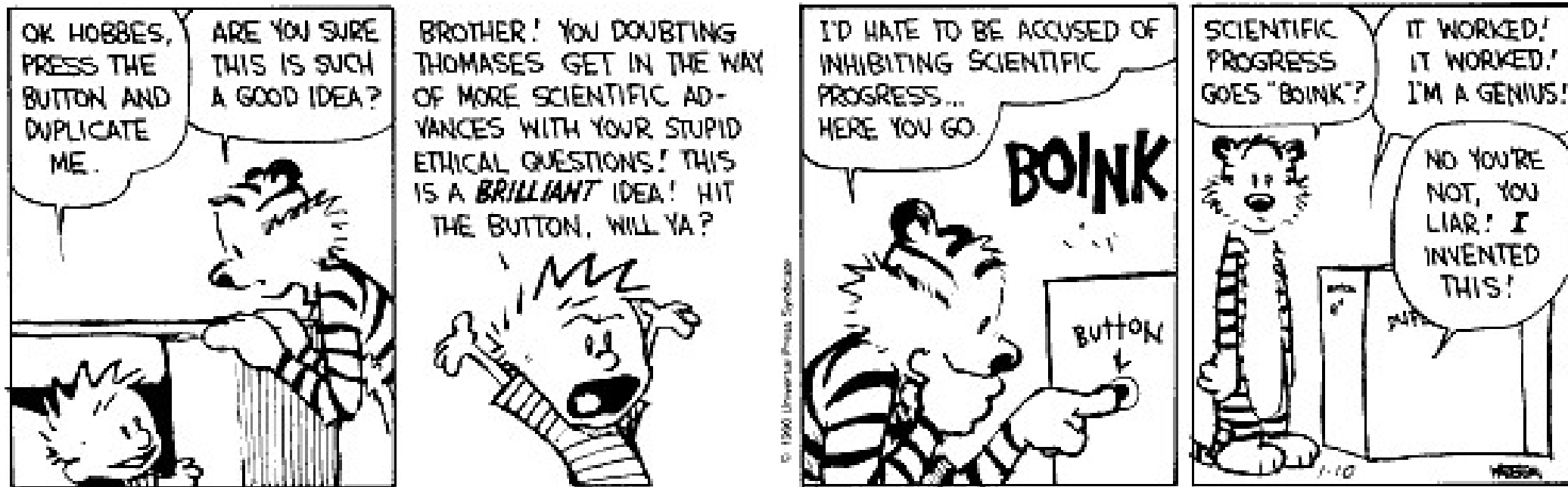
Working set

- Thrashing depends on portion of address space being actively used by each process
 - What do we mean by “actively using”?
- **Working set(T)** = all pages used in last T seconds
 - Larger working set → need more memory to run process efficiently
- **Sum of all working sets should fit in memory**
 - Only run subset of processes that fit in memory

Unix process creation

- System calls to create a process:
 1. `fork()` creates a **copy** of current process (including PC)
 2. `exec(program, args)` **replaces** current address space with specified program
- Why copy parent's address space, then immediately overwrite it?
 - Sometimes, child wants to continue running the same code
 - Allows parent to communicate information to child
 - Allows child to adjust state before starting new program
- Any problems with child being an exact clone of parent?

Cloning



Fork and exec

- `fork()` provides different return code to parent and child
 - Returns 0 to child
 - Returns child's process ID to parent

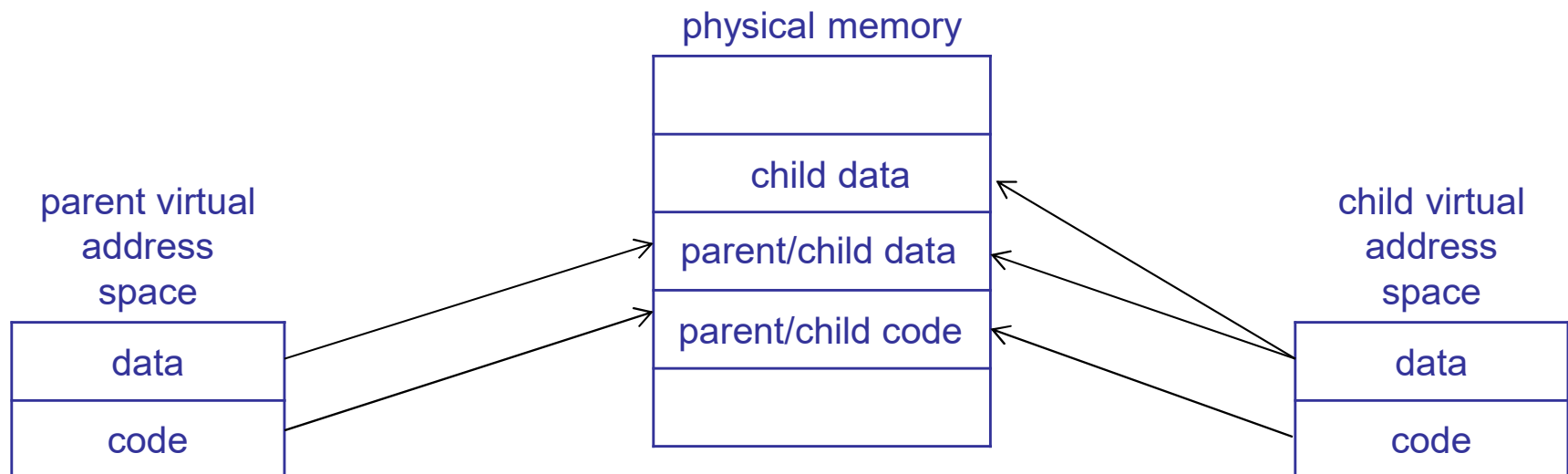
```
if (fork() == 0) {  
    // child  
    exec()  
} else {  
    // parent  
}
```

What does this code do?

```
for (int i=0; i<100; i++) {  
    fork();  
}
```

Avoiding work on fork

- Copying entire address space is expensive
- Instead, make it **look** like it's copied, but do the work later
- This is called **copy-on-write**



Windows process creation

- `CreateProcess(...)`
 - Combines fork and exec
 - Provides options for sharing specific information from parent to child

How to implement a shell?

Autograder bug

```
if (fork() == 0) {  
    exec(command);  
}  
while (child is alive) {  
    if (size of child address space > MAX) {  
        print "process took too much memory";  
        break;  
    }  
}
```

What's wrong with this?